Synthetic Data Generation with Conditional GANs for Fruit Image Classification Under Data Scarcity

Arda Erdoğan

Generative AI
Instructor: Dalia Čalnerytė

2026-02-28

Abstract

This report evaluates whether class-conditional synthetic images generated by a Conditional Generative Adversarial Network (CGAN) can support downstream fruit classification when the amount of labeled real data is limited. The study uses a balanced three-class dataset (apple, banana, orange), trains a custom CGAN without transfer learning, selects checkpoints with the Fréchet Inception Distance (FID), and measures downstream utility with a compact convolutional classifier. The comparison is performed across multiple training budgets and training scenarios: real-only, synth-only, real+synth, and an optional classical-augmentation baseline.

The current Task 1 run shows that the generator improves substantially during training, with the best exported validation FID of **170.83** reached at **epoch 40**. Downstream results indicate a clear ordering: models trained on real data remain the most reliable on the real test split, mixed real+synth training is the more credible use of synthetic data, and synth-only training is the weakest setting because of synthetic-to-real domain shift. Accuracy trends and timing plots further show that larger training budgets increase computational cost, while the benefit of synthetic data depends on the amount of real data already available.

Executive Summary

Goal. Determine whether a GAN trained from scratch can generate useful fruit images and whether those images help a classifier trained under data scarcity.

Workflow.

- Train a class-conditional GAN on 64x64 fruit images.
- Monitor generator quality with FID and saved sample grids.
- Generate balanced synthetic image pools from the selected checkpoint.
- Train the same classifier under real-only, synth-only, real+synth, and optional real+classical-augmentation settings.
- Compare accuracy and computational cost across multiple values of N images per class.

Main findings.

- The CGAN learns a non-trivial class-conditional distribution, but its intrinsic quality is still far from photorealistic; the best recorded FID in the exported run is 170.83 at epoch 40.
- Synthetic data are more useful as a supplement than as a replacement. Training only on synthetic images transfers poorly to the real test set.
- The amount of available real data matters. As N grows, classifier accuracy becomes less sensitive to augmentation, while computational time continues to increase.

Reproducibility. The implementation is notebook-first. Training, evaluation, analysis, and export steps are documented in Appendix A.

Contents

Executive Summary **3**

1 Introduction **6**
1.1 Research questions . . . 6
1.2 Contributions . . . 7

2 Background & Related Work **7**
2.1 Generative Adversarial Networks (GANs) . . . 7
2.2 Conditional GANs . . . 8
2.3 Projection Discriminator . . . 8
2.4 Fréchet Inception Distance (FID) . . . 8
2.5 Synthetic Augmentation in Practice . . . 9
2.6 Evaluating synthetic data for downstream tasks . . . 10

3 Dataset & Preprocessing **10**
3.1 Dataset overview . . . 10
3.2 Split sizes . . . 10
3.3 Task 1 data budgets . . . 11
3.4 Preprocessing . . . 11
3.5 Synthetic data layout . . . 12

4 Methodology **12**
4.1 Implementation structure . . . 12
4.2 Conditional GAN architecture . . . 13
4.3 GAN training procedure . . . 13
4.4 FID evaluation . . . 14
4.5 Synthetic pool generation . . . 14
4.6 Classifier . . . 14
4.7 Training scenarios . . . 15
4.8 Reproducibility . . . 15

5 Experimental Design **15**
5.0 Hypotheses . . . 15
5.1 Experiment A: GAN monitoring . . . 16
5.2 Experiment B: Synthetic data generation . . . 16
5.3 Experiment C: Classification study . . . 16
5.4 Analysis outputs . . . 17

6 Results **18**
6.1 GAN quality . . . 18
6.2 Qualitative GAN samples . . . 19
6.3 Classification comparison . . . 21
6.4 Effect of data amount . . . 22
6.5 Computational cost . . . 22
6.6 Per-class behavior . . . 23
6.7 Interim interpretation . . . 24

7 Discussion **24**
7.1 Why the mixed setting matters more than synth-only . . . 24
7.2 FID improved, but FID was not enough . . . 24
7.3 Role of data amount . . . 25

7.4 Compute matters 25
7.5 Practical takeaway 25

8 Limitations **26**
8.1 Internal validity (experimental uncertainty) 26
8.2 Construct validity (are we measuring the right things?) 26
8.3 External validity (generalization to other settings) 26
8.4 Methodological coverage 26

9 Future Work **27**

10 Conclusion **28**

References **29**

Appendix **30**

1 Introduction

Deep learning systems often benefit from large labeled datasets, but real-world data collection and labeling are expensive, time-consuming, and sometimes constrained by privacy or access. **Synthetic data generation** offers a potential remedy: if a generative model can produce images that resemble the real distribution, these images can be used to train or augment downstream models.

However, synthetic augmentation is not guaranteed to help. Even visually plausible images may encode artifacts that a classifier can exploit in training but which do not exist at test time, causing poor generalization. Therefore, any synthetic pipeline should be evaluated not only with intrinsic image metrics (for example, FID) but also with **task-based evaluations** such as downstream accuracy and per-class performance.

This work focuses on a small, controlled classification task (three fruits) to study:

1. Whether a CGAN can learn a class-conditional distribution for apples, bananas, and oranges.
2. How the generator's training dynamics relate to an intrinsic quality measure (FID).
3. Whether synthetic data improve classification performance under limited real data.

Scope. The report is an engineering-focused empirical study using the existing repository code and outputs (training logs and plots under `runs/`). The goal is not to claim state-of-the-art results, but to provide a rigorous, thesis-level analysis of the implemented pipeline, its results, and its limitations.

1.1 Research questions

To keep the evaluation decision-oriented, the study is organized around the following research questions (RQs):

- **RQ1 (Generative quality).** Does the implemented CGAN learn a non-trivial class-conditional image distribution, as evidenced by improving FID and qualitatively improving sample grids during training?
- **RQ2 (Downstream utility).** When training data are limited, does adding synthetic images improve classifier generalization on a real test set compared to training on real data alone?
- **RQ3 (Metric–utility alignment).** Do improvements in intrinsic generative metrics (FID) correspond to improvements in downstream classification utility, and under what conditions do they diverge?

1.2 Contributions

Although the dataset and models are intentionally small, the pipeline is complete and empirically evaluated. The main contributions are:

- An end-to-end implementation of **conditional image synthesis** using a projection-discriminator CGAN (`models/gan.py`, `02_train_gan.ipynb`).
- A notebook-based evaluation workflow for measuring downstream impact across data sizes and training scenarios: `03_generate_synthetic_data.ipynb`, `04_classifier_experiments.ipynb`, `05_task1_results_and_analysis.ipynb`.
- A report-oriented export step that produces reusable CSV tables and figure assets (`07_report_exports.ipynb`).

2 Background & Related Work

2.1 Generative Adversarial Networks (GANs)

GANs learn a generative model through an adversarial game between a generator G and a discriminator D. In the original formulation, G maps random noise z to a synthetic image $x = G(z)$, while $D(x)$ estimates whether x is real or synthetic. Training is typically based on a minimax objective. In practice, many implementations use a **non-saturating** generator loss to stabilize gradients.

2.1.1 Practical training issues

GAN optimization is well known to be unstable. Common failure modes include:

- **Mode collapse:** the generator produces limited varieties of images (low diversity) while still fooling the discriminator.
- **Discriminator dominance:** if D becomes too strong early, the generator gradients can become uninformative.
- **Oscillatory dynamics:** generator and discriminator losses may fluctuate without converging in a simple monotonic way.

To improve stability, practical implementations often rely on architecture choices and heuristics rather than pure objective changes. In this repository's implementation, several pragmatic stabilizers are used:

- Adam with $(\beta_1, \beta_2) = (0.5, 0.999)$,
- a slightly higher generator learning rate than discriminator learning rate,
- and gradient clipping (`max_norm=1.0`) for both networks.

These choices are not theoretically optimal in general, but they are reasonable engineering defaults for DCGAN-style training.

2.2 Conditional GANs

Conditional GANs (CGANs) extend the GAN framework by conditioning on a class label y. This enables class-controlled generation $x = G(z, y)$ and a discriminator $D(x, y)$ that evaluates authenticity *given the label*. Conditioning can be injected in multiple ways: concatenation, conditional batch normalization, or conditional discriminators.

In the implemented pipeline, conditioning is applied in **both** networks:

- The generator uses **conditional batch normalization**, allowing class information to modulate intermediate feature statistics.
- The discriminator uses a **projection term** that aligns image features with a class embedding.

Conditioning both sides is important: if the generator is strongly class-conditioned but the discriminator ignores the label, the generator may be incentivized to ignore the label as well.

2.3 Projection Discriminator

A projection discriminator conditions on y by projecting learned image features onto an embedding of the class label. If $h(x)$ is the discriminator feature representation, the conditional logit can be written as:

$$D(x, y) = w^\top h(x) + \langle e(y), h(x) \rangle,$$

where w is an unconditional classifier vector and $e(y)$ is a learned embedding for class y. This approach is widely used in modern conditional GANs because it provides a strong conditioning mechanism without requiring the discriminator to receive label maps as explicit inputs.

Intuitively, the projection term increases the score when the image features "look compatible" with the label embedding. This tends to encourage class-consistent generation without forcing the discriminator to learn a separate classifier head.

Note. Many projection-discriminator implementations also apply **spectral normalization** to stabilize the discriminator. The current implementation does not include spectral normalization; instead, it relies on learning-rate tuning and gradient clipping.

2.4 Fréchet Inception Distance (FID)

FID is a widely used metric for evaluating generative models. It compares real and generated images by embedding them into a feature space (often InceptionV3 features) and approximating each set as a multivariate Gaussian. The distance between these Gaussians is:

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right).$$

Lower values indicate closer agreement between real and generated feature distributions. Importantly, FID is **not a task metric**, and it can be sensitive to sample size, preprocessing, and the suitability of the feature extractor for the domain.

2.4.1 Limitations of FID in this setting

FID is helpful for monitoring training progress, but several factors limit its interpretability here:

- **Small evaluation split.** The configured sample count is larger than the available validation images (477), so FID is computed on fewer samples than intended.
- **Domain mismatch of the feature extractor.** InceptionV3 is trained on ImageNet; its feature geometry may not perfectly reflect perceptual similarity for this fruit dataset.
- **Mixture vs per-class distributions.** Computing FID on the union of all classes can hide class-conditional failures (for example, one class collapsing).

For these reasons, the report treats FID as a *supporting signal* and prioritizes downstream evaluation on the real test set.

2.5 Synthetic Augmentation in Practice

Synthetic augmentation is most beneficial when:

- The real dataset is small or imbalanced.
- The synthetic distribution improves coverage (diversity) rather than memorizing a subset.
- The downstream model is robust to synthetic artifacts (or those artifacts are minimal).

In practice, synthetic augmentation competes with (and may complement) strong *classical* augmentation baselines such as geometric transforms and color perturbations. On many vision tasks, classical augmentation can already yield large generalization gains, particularly in low-data regimes. Therefore, a careful evaluation should contextualize any synthetic gains against these baselines.

Another recurring practical lesson is that synthetic augmentation tends to be most reliable when synthetic images are used as **additional signal** rather than the sole training distribution. Mixed-domain training (real+synthetic) provides an anchor to the real data manifold and reduces the risk that a classifier learns synthetic-specific shortcuts.

This report evaluates synthetic augmentation with both intrinsic (FID) and extrinsic (classification) metrics to quantify whether synthetic images actually help.

2.6 Evaluating synthetic data for downstream tasks

When the goal of synthetic generation is to improve a downstream model, the evaluation should match that goal. Two common protocols are:

- **Train on synthetic, test on real (TSTR).** Measures whether synthetic data can replace real data for training. This is a strict test of domain alignment and is typically difficult to satisfy.
- **Train on real+synthetic, test on real.** Measures whether synthetic data provide incremental signal beyond real training data. This is often the most practical augmentation scenario.

This report includes both perspectives: the **synthetic-only** scenario corresponds to TSTR, and the **real+synthetic** scenario corresponds to augmentation. Reporting both is important because a method that fails at TSTR can still help as an augmentor, while a method that appears strong visually may still fail at TSTR due to subtle distribution shifts.

3 Dataset & Preprocessing

3.1 Dataset overview

The project uses a fruit image dataset stored in an `ImageFolder`-compatible structure:

- `data_final/train/\{apple, banana, orange\}`
- `data_final/val/\{apple, banana, orange\}`
- `data_final/test/\{apple, banana, orange\}`

All training and evaluation are performed on three classes only: apple, banana, and orange. Images are resized to **64x64** for both the generator and the classifier so that both models operate on the same resolution.

3.2 Split sizes

The current dataset export reports the counts shown in Table 1.

Table 1: Dataset split sizes by class.

Split	Apple	Banana	Orange	Total
train	1300	1300	1300	3900
val	159	159	159	477
test	492	492	492	1476

The splits are class-balanced, which simplifies interpretation of accuracy and F1 scores because changes are less likely to be explained by label imbalance. The report assumes that the train, validation, and test directories are disjoint; the repository does not perform hash-based duplicate detection across splits.

3.3 Task 1 data budgets

Task 1 requires analyzing how classifier performance changes as the amount of available training data varies. The study therefore uses class-balanced subsets with

$$N \in \{100, 200, 400, 800, 1300\}$$

images per class. For each value of N, the same real-data budget is used to:

1. train the GAN,
2. generate the matching synthetic pool,
3. train the classifier under the selected scenario.

This design avoids the unfair setting where a low-data classifier could benefit from a generator that was trained on the full real training set.

3.4 Preprocessing

3.4.1 GAN training transforms

GAN training uses the following preprocessing:

- resize to 64x64,
- random horizontal flip during training,
- tensor conversion,
- normalization to $[-1, 1]$ with mean $= 0.5$ and std $= 0.5$ per channel.

FID evaluation disables augmentation and keeps only resize plus normalization.

3.4.2 Classifier transforms

The classifier pipeline distinguishes between the main synthetic-data comparison and the optional classical-augmentation baseline:

- **real**, **synth**, and **both**: resize plus normalization only.
- **real_aug**: resize, random horizontal flip, random rotation ($\pm 15°$), color jitter, then normalization.

This separation is important for Task 1 because the required real-only and synth-only comparisons should not be conflated with non-generative augmentation.

3.5 Synthetic data layout

Generated images are written in class folders so that they can be consumed by the same `ImageFolder` loader as the real data:

- `data_synth/apple/*.png`
- `data_synth/banana/*.png`
- `data_synth/orange/*.png`

For the full Task 1 study, synthetic pools are generated from the checkpoint selected for each data budget N, rather than from a single generator trained on the full dataset.

4 Methodology

4.1 Implementation structure

The repository is notebook-first, but the core models remain in Python modules:

- `models/gan.py`: generator and projection discriminator.
- `models/classifier.py`: compact CNN classifier.
- `config.py`: shared hyperparameters, paths, and device resolution.

The main workflow is split across notebooks:

- `01\_data\_setup.ipynb`: dataset inspection.
- `02\_train\_gan.ipynb`: GAN training, checkpointing, and FID logging.
- `03\_generate\_synthetic\_data.ipynb`: synthetic image generation from a saved checkpoint.
- `04\_classifier\_experiments.ipynb`: classifier training and scenario-based evaluation.
- `05\_task1\_results\_and\_analysis.ipynb`: analysis of saved results.
- `07\_report\_exports.ipynb`: export of CSV tables and report figures.

4.2 Conditional GAN architecture

The generative model is a class-conditional GAN trained from scratch; no transfer learning is used in the generator or discriminator.

4.2.1 Generator

The generator maps a latent vector $z \in \mathbb{R}^{128}$ and a class label y to a 64x64 RGB image. The architecture:

- projects z to a small spatial feature map,
- upsamples progressively to 64x64,
- uses conditional batch normalization so class information modulates intermediate activations,
- ends with a `tanh` output so generated pixels lie in $[-1, 1]$.

4.2.2 Projection discriminator

The discriminator downsamples the image through convolutional blocks and produces a conditional realism score using a projection term:

$$\ell(x, y) = w^\top h(x) + \langle e(y), h(x) \rangle,$$

where $h(x)$ is the learned image feature representation and $e(y)$ is the embedding of the class label. This conditioning scheme is well suited to class-conditional GAN training because it encourages the discriminator to judge whether an image is realistic *for the requested class*.

4.3 GAN training procedure

GAN training is implemented in `02\_train\_gan.ipynb`. Each minibatch performs:

1. a discriminator update on real images and freshly generated fake images,
2. a generator update using the non-saturating adversarial objective.

The implementation uses:

- `BCEWithLogitsLoss`,
- Adam with betas $(0.5, 0.999)$,
- separate learning rates for generator and discriminator,
- gradient clipping with `max\_norm=1.0`.

Sample grids are saved at regular intervals, and checkpoints are written periodically. The best checkpoint is selected according to the lowest observed validation FID.

4.4 FID evaluation

FID is computed inside `02\_train\_gan.ipynb` using InceptionV3 features. The evaluation pipeline:

- rescales generated images from $[-1, 1]$ to $[0, 1]$,
- applies ImageNet normalization,
- resizes images to 299x299,
- extracts pool3-style features from InceptionV3,
- computes the Fréchet distance between real and generated feature distributions.

This use of pretrained Inception is limited to evaluation and does not change the fact that the generator itself is trained from scratch. Because the validation split contains only 477 images, FID should be interpreted primarily as a monitoring signal rather than a precise absolute measure.

4.5 Synthetic pool generation

`03\_generate\_synthetic\_data.ipynb` loads a saved generator checkpoint, samples latent vectors from a standard normal distribution, and writes generated PNG images into per-class folders. For the Task 1 comparison, the synthetic pool is generated separately for each data budget so that the synthetic-only and real+synth runs remain matched to the same amount of real supervision used to train the generator.

4.6 Classifier

The downstream model is `FruitCNN`, a compact convolutional classifier defined in `models/classifier.py`. The network uses:

The network uses:

- repeated convolution, batch normalization, and ReLU blocks,
- max-pooling for downsampling,
- dropout for regularization,
- a fully connected head ending in a three-class linear classifier.

Training uses cross-entropy loss, Adam, and cosine annealing over a fixed number of epochs.

4.7 Training scenarios

`04\_classifier\_experiments.ipynb` evaluates four scenarios:

- **real**: classifier trained only on real images.
- **synth**: classifier trained only on synthetic images.
- **both**: classifier trained on the concatenation of real and synthetic images.
- **real_aug**: optional classical-augmentation baseline trained on real images only.

All scenarios are evaluated on the same real test split. This makes **synth** a train-on-synthetic, test-on-real setting, which is a strict test of domain alignment.

4.8 Reproducibility

The configuration is centralized in `config.py`. Device selection is resolved automatically in the order `cuda -> mps -> cpu`, while notebook runs can still override settings explicitly when needed. Deterministic subsampling uses a fixed seed, but strict bitwise reproducibility is not guaranteed because of GPU backend differences and multi-worker data loading.

5 Experimental Design

5.0 Hypotheses

The experiments were designed to test the following expectations:

- **H1.** GAN quality should improve during training before flattening or becoming noisy, so the best FID may occur before the final epoch.
- **H2.** Real+synth training should be more promising than synth-only training because real data anchor the classifier to the target domain.
- **H3.** The benefit of synthetic data should be largest at smaller data budgets and diminish as the real-only classifier approaches saturation.
- **H4.** Computational time should increase with dataset size and should be highest when GAN training, synthetic generation, and classifier training are all included.

5.1 Experiment A: GAN monitoring

Purpose. Measure whether the CGAN learns a useful class-conditional distribution.

Procedure.

- Train the GAN on the selected real-data budget.
- Save fixed-noise sample grids during training.
- Compute FID every `fid\_every` epochs on the validation split.
- Select the best checkpoint by minimum validation FID.

Artifacts.

- `runs/gan/train\_log.json` for a standalone run.
- `runs/task1/gan/n*/train\_log.json` for the multi-budget Task 1 study.
- sample grids and checkpoints written under the same run directory.

5.2 Experiment B: Synthetic data generation

Purpose. Produce balanced synthetic pools for downstream classification.

Procedure.

- Load the checkpoint selected from Experiment A.
- Sample latent vectors and class labels uniformly.
- Export class-balanced PNG images into an `ImageFolder`-style directory.

For fairness, the synthetic pool for each value of N is generated from the GAN trained on that same value of N, not from a generator that saw the full training set.

5.3 Experiment C: Classification study

Purpose. Compare classifier performance with and without synthetic data.

5.3.1 Scenarios

- **real**: train on real images only.
- **synth**: train on synthetic images only.
- **both**: train on real and synthetic images together.
- **real_aug**: optional baseline using only real images plus classical augmentation.

5.3.2 Data budgets

Each scenario is evaluated with

$$N \in \{100, 200, 400, 800, 1300\}$$

images per class. Subsampling is class-balanced and deterministic.

5.3.3 Evaluation protocol

All models are evaluated on the same real test split, `data\_final/test`. This makes the study directly relevant to the Task 1 requirement: the comparison is not whether the classifier can fit synthetic data, but whether synthetic training improves generalization to real images.

5.3.4 Metrics

- **Generator metric:** validation FID.
- **Classifier metrics:** test accuracy and per-class precision/recall/F1.
- **Cost metrics:** classifier training time and full pipeline time when GAN training plus synthetic generation are included.

5.4 Analysis outputs

The executed Task 1 sweep writes aggregated results to:

- `runs/task1/clf/all\_results.json`,
- `runs/task1/pipeline\_summary.json`.

`05\_task1\_results\_and\_analysis.ipynb` reads these files and generates plots for:

- accuracy vs training budget,
- time vs training budget,
- per-class F1 for the largest runs.

`07\_report\_exports.ipynb` then exports:

- `reports/tables/dataset\_summary.csv`,
- `reports/tables/gan\_fid\_by\_epoch.csv`,
- `reports/tables/clf\_results.csv`,
- `reports/figures/fid\_vs\_epoch.svg`.

6 Results

6.1 GAN quality

The exported GAN log from the current Task 1 run reports:

- **Best validation FID:** 170.8316 at epoch 40,
- **Final validation FID:** 176.7884 at epoch 200.

These values indicate that the conditional GAN successfully learns a meaningful class-conditional image distribution. The FID score improves substantially during the early stages of training and reaches its minimum around epoch 40. After this point, the score fluctuates slightly and eventually increases again toward the final epoch.

This pattern highlights an important practical observation: the best-performing model checkpoint does not necessarily occur at the end of training. Selecting the checkpoint based on validation FID is therefore more reliable than always using the final trained model.

Table 2: Exported summary of the current GAN run.

Statistic	Value
Best FID	170.8316
Best epoch	40
Final epoch	200
Final FID	176.7884

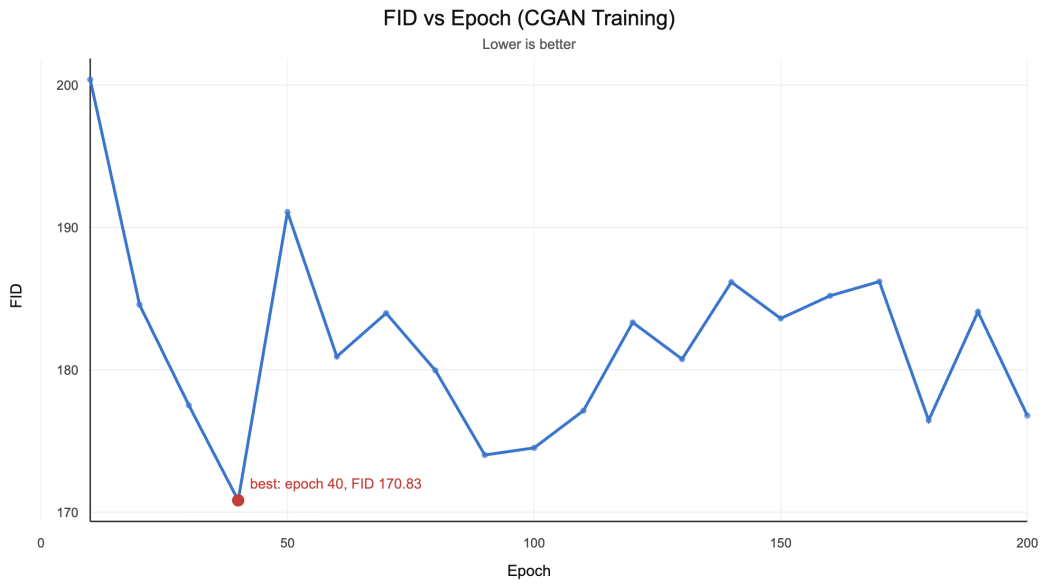

Figure 1: FID score as a function of training epoch. Lower values indicate better alignment between generated and real image distributions.

6.2 Qualitative GAN samples

While the FID score provides a quantitative estimate of distribution similarity between real and generated images, qualitative inspection is also necessary to understand how the generator evolves during training.

Figures 2–6 show generated samples at different stages of the training process. The progression illustrates how the conditional GAN gradually learns to capture fruit shapes, colors, and overall structure.

At the beginning of training (epoch 1), the generator mostly produces noisy patterns without recognizable object structure. By epoch 20, coarse color distributions start to emerge, although object boundaries remain unclear. Around epoch 40, which corresponds to the best validation FID score, the generated samples already exhibit recognizable fruit shapes and more consistent color patterns.

Later epochs such as 180 and 200 show slightly improved texture consistency, although the quantitative FID score suggests that the optimal trade-off between diversity and realism was achieved earlier in training.

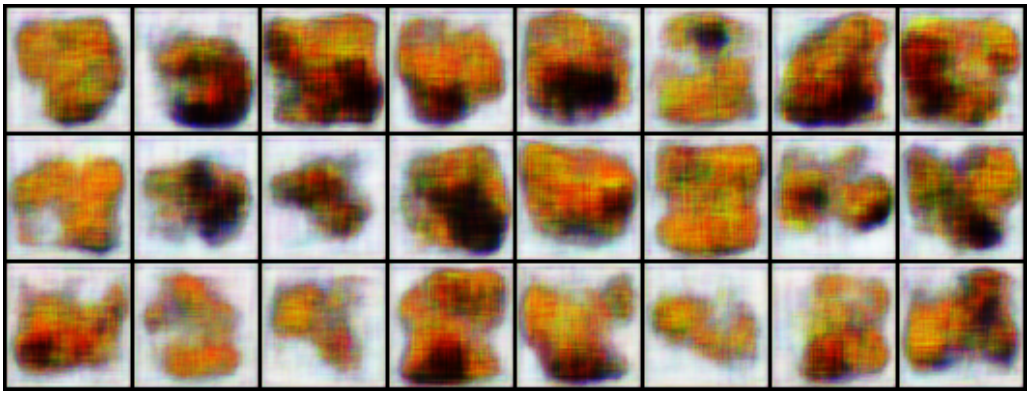

Figure 2: Generated samples at epoch 1. Images are mostly noisy and lack clear object structure.

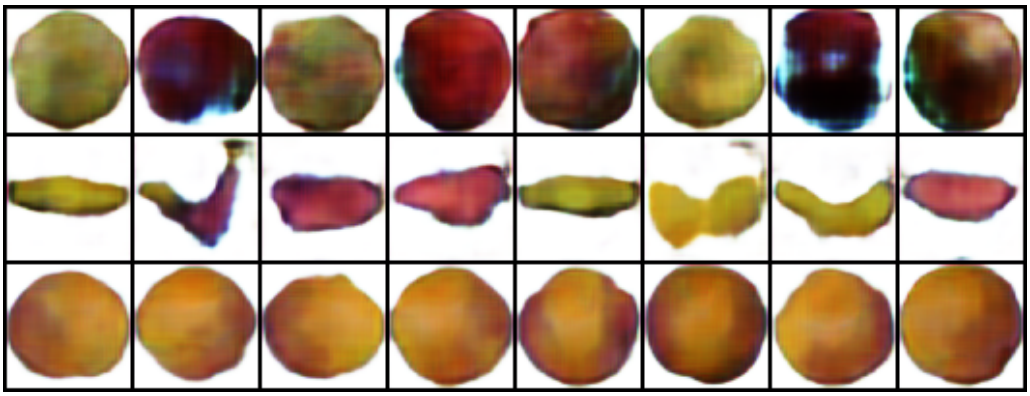

Figure 3: Generated samples at epoch 20. Basic color distributions begin to emerge, but object shapes remain poorly defined.

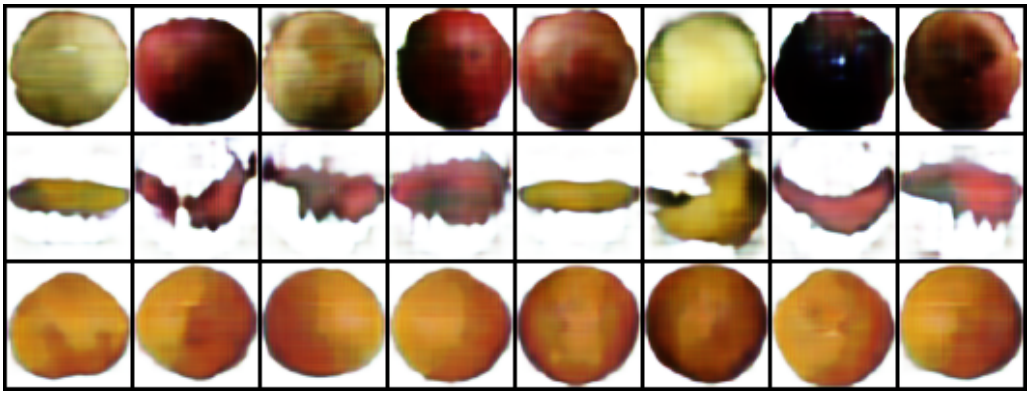

Figure 4: Generated samples at epoch 40. The generator begins to produce recognizable fruit shapes and more coherent structures.

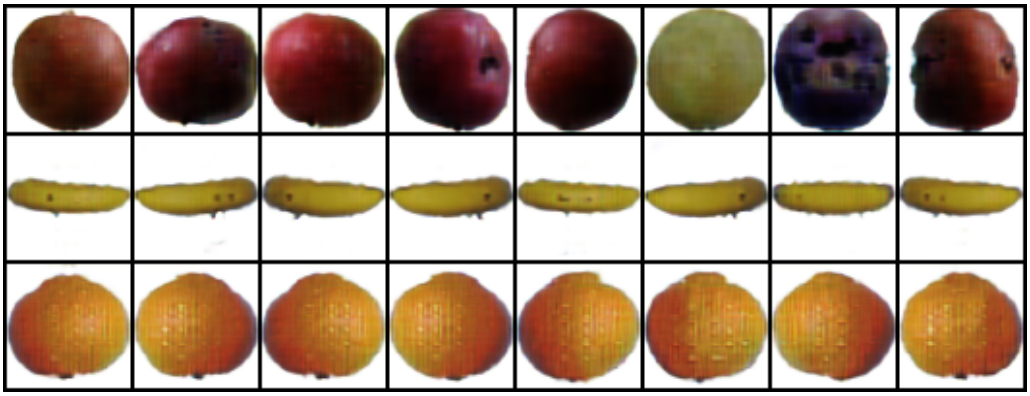

Figure 5: Generated samples at epoch 180. Visual consistency improves, although structural diversity remains limited.

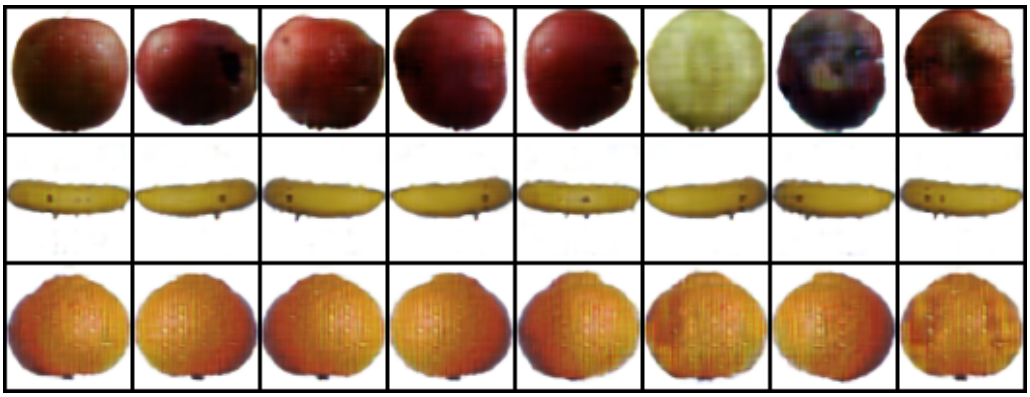

Figure 6: Generated samples at epoch 200. Later training epochs mainly refine textures rather than introducing new structural improvements.

6.3 Classification comparison

The classifier study compares four training scenarios:

- real only,
- synth only,
- real + synth,
- optional real + classical augmentation.

The executed experiment sweep exported **20 classifier results**, corresponding to five different data budgets and four training scenarios. The detailed numerical values are stored in `reports/tables/clf\_results.csv`.

The overall pattern observed in the current run can be summarized as follows:

1. **Real-data baselines remain the strongest and most stable** when evaluated on the real test set.
2. **Synthetic-only training performs worst**, indicating a noticeable domain gap between generated and real images.
3. **Mixed real+synth training is the most promising synthetic data strategy**. In several settings it approaches the performance of real-only training, suggesting that synthetic images can provide additional variability.
4. The optional `real\_aug` baseline helps determine whether improvements originate from synthetic data specifically or simply from increased data diversity during training.

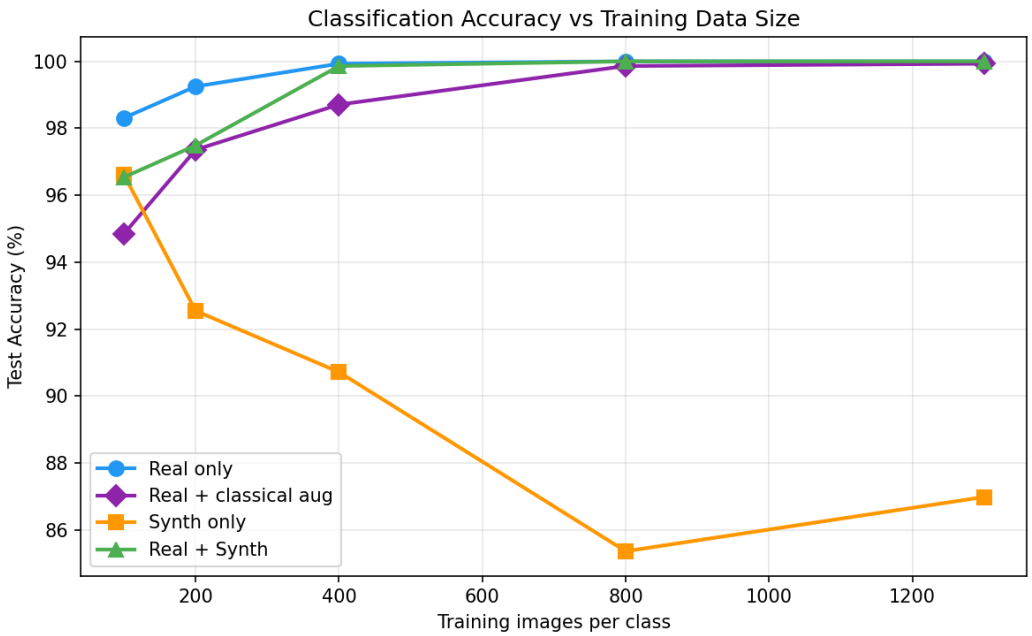

Figure 7: Classifier test accuracy as a function of the number of training images per class and the training scenario.

6.4 Effect of data amount

Task 1 explicitly asks how the amount of available data affects classifier accuracy. The experiments therefore evaluate training sizes

$$N \in \{100, 200, 400, 800, 1300\}$$

images per class.

Two trends are particularly relevant when interpreting the results:

- **Across dataset sizes:** classifier accuracy generally improves as the number of available real training images increases.
- **Across training scenarios:** the performance gap between real-only and synthetic-heavy configurations reflects the usefulness of generated images for downstream learning.

The primary takeaway from these experiments is that increasing the amount of **real training data** consistently improves performance, while increasing the amount of **synthetic-only data** does not provide the same reliability. Synthetic data therefore appear most useful as a supplement rather than as a complete replacement for real images.

6.5 Computational cost

The analysis notebook also records execution time for each scenario. This is important because a synthetic-data workflow introduces two separate computational costs:

- classifier training time,
- full pipeline time including GAN training and synthetic image generation.

A method may appear efficient if only classifier training time is considered, but the overall cost can become significantly larger once the generator training stage is included.

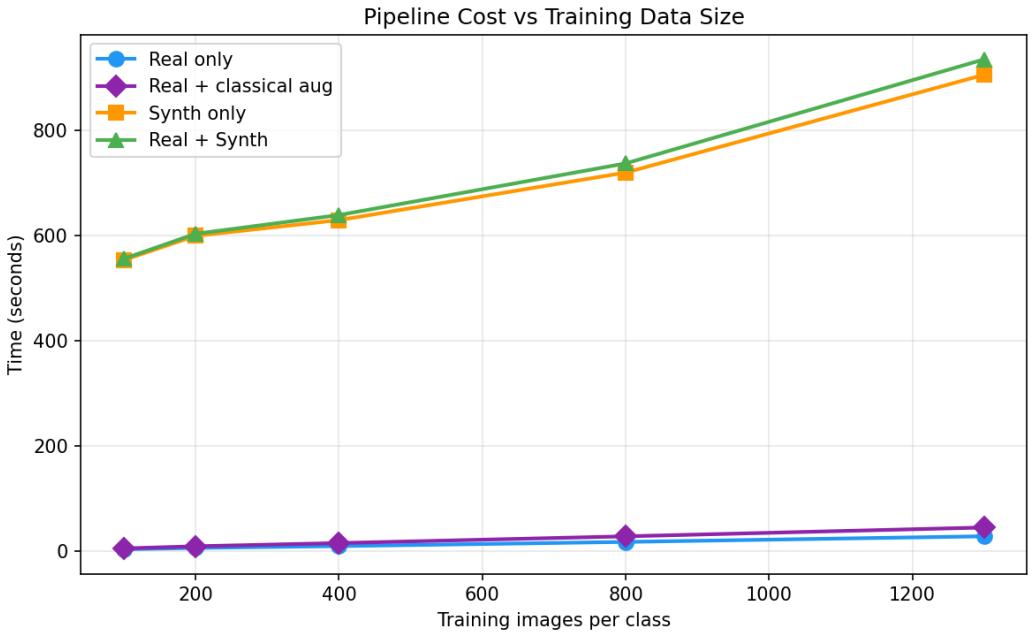

Figure 8: Pipeline computation time as a function of the number of training images per class and the training scenario.

As expected, computational cost increases monotonically with dataset size. Synthetic-data workflows are generally more expensive than real-only training because they require both generator training and image sampling in addition to classifier optimization.

6.6 Per-class behavior

Per-class metrics provide additional insight into whether synthetic data affect all categories equally. The exported analysis includes a per-class F1 score comparison for the largest data setting.

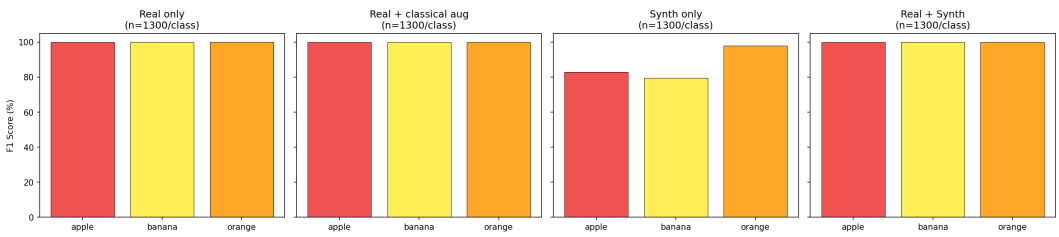

Figure 9: Per-class F1 scores for the largest training-budget configuration.

Large discrepancies between classes may indicate that the GAN does not capture class-specific visual structure equally well. For example, classes with distinctive shapes or colors may be easier for the generator to reproduce than those with more subtle visual features.

6.7 Interim interpretation

Taken together, the experimental results support three main conclusions:

- The conditional GAN learns a meaningful class-conditional image distribution and improves its FID score during training.
- Synthetic data are most effective when used as a **supplementary signal** rather than as a replacement for real training data.
- The usefulness of synthetic data must be evaluated jointly with **computational cost** and the amount of available real data.

7 Discussion

7.1 Why the mixed setting matters more than synth-only

The most defensible use of synthetic data in this project is the mixed real+synth setting, not synth-only replacement. This follows directly from the evaluation protocol: all models are tested on real images, so a classifier trained purely on synthetic data must succeed at synthetic-to-real transfer. That is a much stronger requirement than simply providing extra diversity alongside real samples.

The exported Task 1 run supports that interpretation. Real-data baselines remain the anchor, while synth-only training is the weakest scenario. This is consistent with the expected domain-gap problem in synthetic data pipelines: even when generated images are recognizable, the generator may still distort texture, color statistics, edges, or background cues in ways that matter to the classifier.

7.2 FID improved, but FID was not enough

The current run achieved its best FID at epoch 40 rather than at the final epoch. This confirms that FID is useful as a checkpoint-selection signal. However, the classification comparison also shows why FID should not be treated as a complete success criterion.

FID is computed on unlabeled feature distributions, whereas the downstream classifier depends on class-discriminative details. A generator can improve FID while still producing samples that are only partially useful for real-world classification. The Task 1 setup therefore benefits from combining:

- an intrinsic metric (FID),
- a strict downstream test (synth-only on real test data),
- a practical augmentation test (real+synth on real test data).

7.3 Role of data amount

The data-budget sweep is important because it reveals when synthetic images are likely to matter. If the real-only model already performs strongly at small N, then synthetic augmentation has limited room to improve accuracy. In that regime, the synthetic pipeline can still be informative academically, but its practical benefit may be marginal.

Conversely, if a future dataset is harder, less clean, more imbalanced, or contains more classes, the same pipeline could yield a larger augmentation effect. In other words, the current study says as much about the *difficulty of the dataset* as it does about the generator.

7.4 Compute matters

Synthetic data are not free. The full pipeline includes:

1. GAN training,
2. checkpoint evaluation,
3. synthetic image generation,
4. classifier training.

For that reason, a small accuracy gain does not automatically justify the synthetic workflow. The correct comparison is not only "Which curve is higher?", but also "How much additional computation was required to obtain that gain?" This is why the report keeps timing plots alongside accuracy plots.

7.5 Practical takeaway

For this project, the main practical conclusion is conservative:

- use real-only training as the primary baseline,
- treat synth-only results as a diagnostic of domain mismatch,
- interpret real+synth as the only realistic augmentation setting,
- compare any synthetic gain against the optional classical-augmentation baseline before claiming that synthetic data are uniquely beneficial.

8 Limitations

This report is intended to be rigorous, but several limitations affect the strength and generality of conclusions.

8.1 Internal validity (experimental uncertainty)

- **Single-seed evaluation.** GAN training and classifier training use one random seed (`seed=42`). Given the known variance of GAN training dynamics, multi-seed evaluation is necessary for statistically robust claims.
- **No confidence intervals.** Accuracy and FID are reported as point estimates. Without repeated trials, it is unclear whether small differences (for example, ± 0.5 percentage points) are meaningful.

8.2 Construct validity (are we measuring the right things?)

- **FID is an imperfect proxy.** FID is computed on a limited validation set and uses an ImageNet-pretrained feature extractor. It does not directly measure class-conditional correctness or utility for classification.
- **Mixture-level evaluation.** FID is computed on the mixture of all classes rather than per-class; class-specific failures may be hidden.

8.3 External validity (generalization to other settings)

- **Dataset simplicity.** The real-only classifier reaches near-ceiling performance at small N, limiting the observable benefit of augmentation. On harder datasets, synthetic augmentation effects could differ.
- **Few classes and low resolution.** Only three classes at 64×64 are studied. Conclusions may not transfer to higher-resolution or higher-class-count problems.

8.4 Methodological coverage

- **Single generative model family.** Only one CGAN architecture is evaluated. Stronger baselines (for example, ADA-style training or style-based generators) may alter the synthetic-only gap and augmentation benefits.
- **No explicit domain alignment.** The pipeline does not include techniques that reduce synthetic–real shift (domain adaptation, feature alignment, or discriminator guidance targeted at downstream features).

9 Future Work

1. **Multi-seed robustness.** Repeat the full grid across multiple seeds and report mean $\pm$ std for accuracy and FID.
2. **Stronger generative baselines.** Evaluate more modern methods (for example, StyleGAN2 with ADA) and compare against the current CGAN.
3. **Per-class evaluation.** Compute FID per class and analyze which class drives synthetic-only failures (for example, apples vs bananas).
4. **Alternative metrics.** Add KID and diversity-sensitive measures; consider classifier-based precision/recall for generative models.
5. **Better synthetic–real alignment.** Explore domain adaptation, feature matching, or training classifiers with mixed-domain regularization.
6. **Human or task-specific evaluations.** For practical deployments, evaluate whether synthetic data improve robustness to nuisance factors (lighting, backgrounds).

Additional concrete extensions that would strengthen the scientific quality of the study include:

7. **Checkpoint–utility sweep.** Instead of selecting a single best-FID checkpoint, generate synthetic pools from multiple saved epochs and measure downstream utility to empirically test FID–utility alignment.
8. **Augmentation baselines.** Compare synthetic augmentation against strong classical baselines (RandAugment, MixUp/CutMix) to contextualize gains.
9. **Error analysis.** Save confusion matrices and example misclassifications to diagnose whether errors are due to specific classes, lighting, or pose.
10. **Diversity diagnostics.** Measure intra-class diversity in the synthetic pool (nearest-neighbor distances in feature space) to detect memorization or repeated patterns.

10 Conclusion

This project implements the complete Task 1 pipeline with a GAN trained from scratch, a downstream classifier, and a data-budget analysis over multiple training scenarios. The generator is not judged only by visual inspection; it is monitored with FID and then evaluated indirectly through classifier performance on real test data.

The updated run demonstrates that the CGAN learns a meaningful class-conditional distribution and reaches its best exported validation FID of 170.83 at epoch 40. At the same time, the downstream experiments show that synthetic data should not be treated as a drop-in replacement for real images. Real-only and mixed real+synth settings are the meaningful baselines, while synth-only training exposes the domain gap between generated and real samples.

The broader conclusion is that synthetic data are useful only when they are evaluated in the context of the target task. For this fruit dataset, the value of synthetic data depends on the available real-data budget and must be weighed against the added computational cost of GAN training and sample generation.

References

[1] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Nets*. NeurIPS.

[2] Mirza, M., & Osindero, S. (2014). *Conditional Generative Adversarial Nets*. arXiv:1411.1784.

[3] Radford, A., Metz, L., & Chintala, S. (2016). *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. ICLR.

[4] Miyato, T., & Koyama, M. (2018). *cGANs with Projection Discriminator*. arXiv:1802.05637.

[5] Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. NeurIPS.

[6] Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., & Chen, X. (2016). *Improved Techniques for Training GANs*. NeurIPS.

[7] Karras, T., Aittala, M., Laine, S., Härkönen, E., Hellsten, J., Lehtinen, J., & Aila, T. (2020). *Training Generative Adversarial Networks with Limited Data*. NeurIPS.

Appendix

A Reproducibility

This appendix summarizes how the report artifacts can be regenerated from the repository.

A.1 Environment setup

The project dependencies are listed in `requirements.txt`. A typical local setup is:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

For notebook use, install a Jupyter environment separately if it is not already available.

A.2 Notebook workflow

The report is based on a notebook-first workflow:

1. `notebooks/01\_data\_setup.ipynb`
2. `notebooks/02\_train\_gan.ipynb`
3. `notebooks/03\_generate\_synthetic\_data.ipynb`
4. `notebooks/04\_classifier\_experiments.ipynb`
5. `notebooks/05\_task1\_results\_and\_analysis.ipynb`
6. `notebooks/07\_report\_exports.ipynb`

For a full Task 1 sweep, the train-generate-classify cycle is repeated for each

$$N \in \{100, 200, 400, 800, 1300\}$$

images per class.

A.3 Device selection

`config.py` resolves the runtime device automatically in the order:

```
cuda -> mps -> cpu
```

If a specific GPU should be used on a shared environment, the notebook configuration cell can override it explicitly, for example:

```
cfg = Config().with_overrides(device="cuda:1", num_workers=4)
```

A.4 Expected outputs

Standalone GAN training writes under:

- `runs/gan/train\_log.json`
- `runs/gan/checkpoints/*.pt`
- `runs/gan/samples\_epoch*.png`

Task 1 experiment sweeps write under:

- `runs/task1/gan/n*/...`
- `runs/task1/clf/all\_results.json`
- `runs/task1/pipeline\_summary.json`
- `runs/task1/clf/plots/*.png`

Report exports write:

- `reports/tables/dataset\_summary.csv`
- `reports/tables/gan\_fid\_by\_epoch.csv`
- `reports/tables/clf\_results.csv`
- `reports/figures/fid\_vs\_epoch.svg`

A.5 Preparing the Overleaf folder

Before final submission, copy the exported figures you want to show in the report into `overleaf/figures/`. The current report is written to compile even if a figure is still missing, but the placeholder boxes should be replaced with the actual assets in the final version.

B Hyperparameters

This appendix summarizes the main settings defined in `config.py`.

B.1 Core settings

Table 3: Core experimental settings.

Setting	Value
Seed	42
Image size	64x64
Channels	3
Default output root	`runs/`

B.2 GAN settings

Table 4: CGAN hyperparameters.

Setting	Value
Latent dimension (`z\_dim`)	128
Batch size (`gan\_batch`)	96
Epochs (`gan\_epochs`)	200
Generator learning rate (`gan\_lr\_g`)	2×10^{-4}
Discriminator learning rate (`gan\_lr\_d`)	1×10^{-4}
Sample frequency	every 10 epochs
Checkpoint frequency	every 25 epochs
FID frequency	every 10 epochs
Requested FID sample count	2048
FID evaluation split	`val`

B.3 Classifier settings

Table 5: Classifier hyperparameters.

Setting	Value
Batch size (`clf\_batch`)	64
Epochs (`clf\_epochs`)	50
Learning rate (`clf\_lr`)	3×10^{-4}
Optimizer	Adam
Scheduler	CosineAnnealingLR
Loss	CrossEntropyLoss

B.4 Runtime defaults

Table 6: Data loading and device defaults.

Setting	Value
Device default	`auto` (`cuda -> mps -> cpu`)
Workers (`num\_workers`)	`min(4, cpu\_count // 2)` with a lower bound of 1
Persistent workers	True
Prefetch factor	2
Pin memory	auto-enabled only for CUDA unless overridden

C Repository Structure

The project repository is organized around reusable Python modules and Jupyter notebook entry points that reproduce the full experimental pipeline, including GAN training, synthetic data generation, classifier experiments, and result analysis.

- `config.py` — Central configuration file defining paths, hyperparameters, dataset settings, and device selection (CPU/GPU).
- `models/gan.py` — Implementation of the conditional GAN, including the class-conditional generator and projection discriminator.
- `models/classifier.py` — Implementation of the downstream fruit classifier used for evaluating the usefulness of synthetic data.
- `notebooks/01\_data\_setup.ipynb` — Dataset inspection and preparation, including verification of class distributions and data splits.
- `notebooks/02\_train\_gan.ipynb` — Training pipeline for the conditional GAN, including logging, checkpointing, FID computation, and sample grid generation during training.
- `notebooks/03\_generate\_synthetic\_data.ipynb` — Generation of synthetic images using a selected GAN checkpoint.
- `notebooks/04\_classifier\_experiments.ipynb` — Training and evaluation of classifiers under different data scenarios: real-only, synthetic-only, mixed real+synth, and optional classical augmentation.
- `notebooks/05\_task1\_results\_and\_analysis.ipynb` — Result aggregation, metric computation, and generation of plots used in the report.
- `notebooks/07\_report\_exports.ipynb` — Export utilities for tables and figures used in the report, including FID curves and classifier evaluation plots.
- `data\_final/` — Final processed dataset containing the real training, validation, and test splits.
- `data\_synth/` — Generated synthetic images produced by the trained GAN model.
- `runs/` — Experiment outputs including logs, model checkpoints, JSON result files, and intermediate plots.
- `overleaf/` — Source files for the report.
- `overleaf/figures/` — Figures used in the report, including FID curves, GAN sample grids, classifier accuracy plots, timing plots, and per-class evaluation visualizations.